ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC B ÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO THÍ NGHIỆM

LAB 01: THIẾT KẾ MẠCH RTL

Môn: Cơ Sở Thiết Kế Vi Mạch

MSSV	Họ & Tên
2310434	Lâm Chí Dĩnh
2313225	Trần Đình Thiên
2313014	Thái Tài

GVHD: Thầy Nguyễn Trung Hiếu

Thành phố Hồ Chí Minh, ngày 24 tháng 10 năm 2025

Mục lục

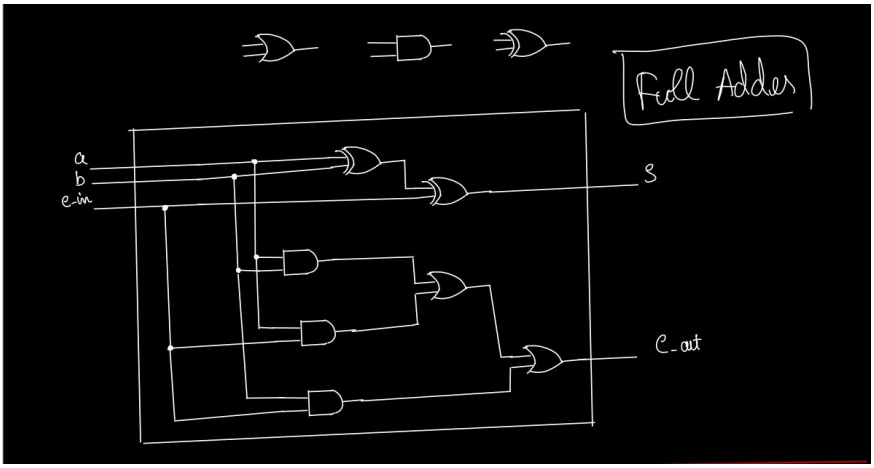
THÍ NGHIỆM 1: 3
1.1. Thiết kế mạch logic: 3
1.2. Hoạt động của mạch 4
1.2.1. Khối thanh ghi (Register blocks): 4
1.2.2. Khối cộng 8-bit (8-bit adder): 4
1.2.3. Khối logic (Logic circuit): 5
1.2.4. Khối đầu ra (Output blocks): 5
1.3. Code RTL (SystemVerilog) 5
1.4. Kiểm định thiết kế: 8
1.4.1. Các giả định: 8
1.4.2. Kết quả kiểm định: 9
THÍ NGHIỆM 2: 11
2.1. Thiết kế mạch logic: 11
2.2. Hoạt động của mạch 11
2.2.1. Tín hiệu điều khiển i_add_sub: 11
2.2.2. Cải tiến Khối cộng 8-bit: 11
2.2.3. Cải tiến Khối logic: 12
2.3. Code RTL (SystemVerilog) 12
2.4. Kiểm định thiết kế: 14
2.4.1. Các giả định: 14
2.4.2. Kết quả kiểm định: 15
THÍ NGHIỆM 3: 18
3.1. Thiết kế mạch logic: 18
3.2. Hoạt động của mạch 19
3.3. Code RTL (SystemVerilog) 19
3.4. Kiểm định thiết kế: 21
3.4.1. Các giả định: 21
3.4.2. Kết quả kiểm định: 21
THÍ NGHIỆM 4: 24
4.1. Thiết kế mạch logic: 24
4.2. Hoạt động của mạch 25
4.3. Code RTL (SystemVerilog) 26
4.4. Kiểm định thiết kế: 29
4.4.1. Các giả định: 29
4.4.2. Kết quả kiểm định: 30

THÍ NGHIỆM 1:

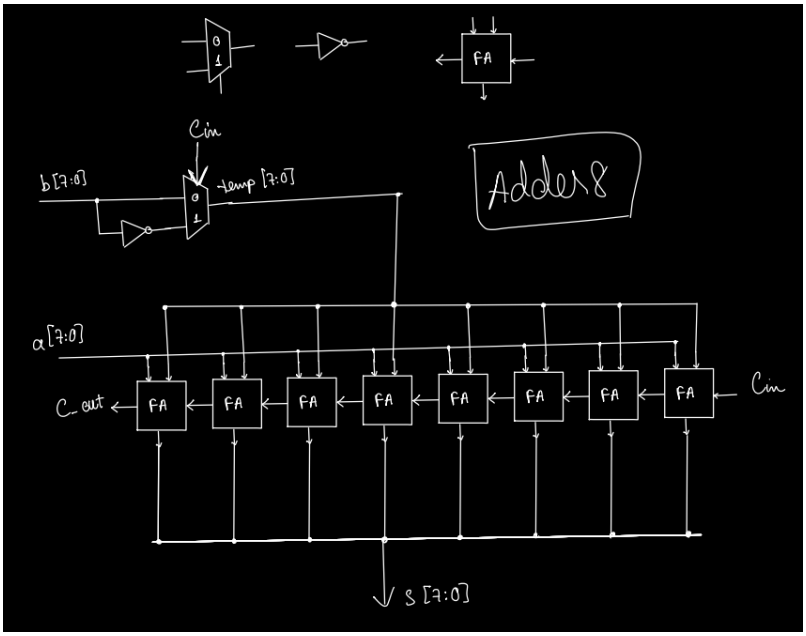
Objective: Know how to program a system to add the value of an input A to itself repeatedly.

Description: The circuit in Figure 1, which is often called an accumulator, is used to add the value of an input A to itself repeatedly. The circuit includes a carry out from the adder, as well as an overflow output signal. If the input A is considered as a 2's-complement number, then overflow should be set to 1 in the case where the output sum produced does not represent a correct 2'scomplement result

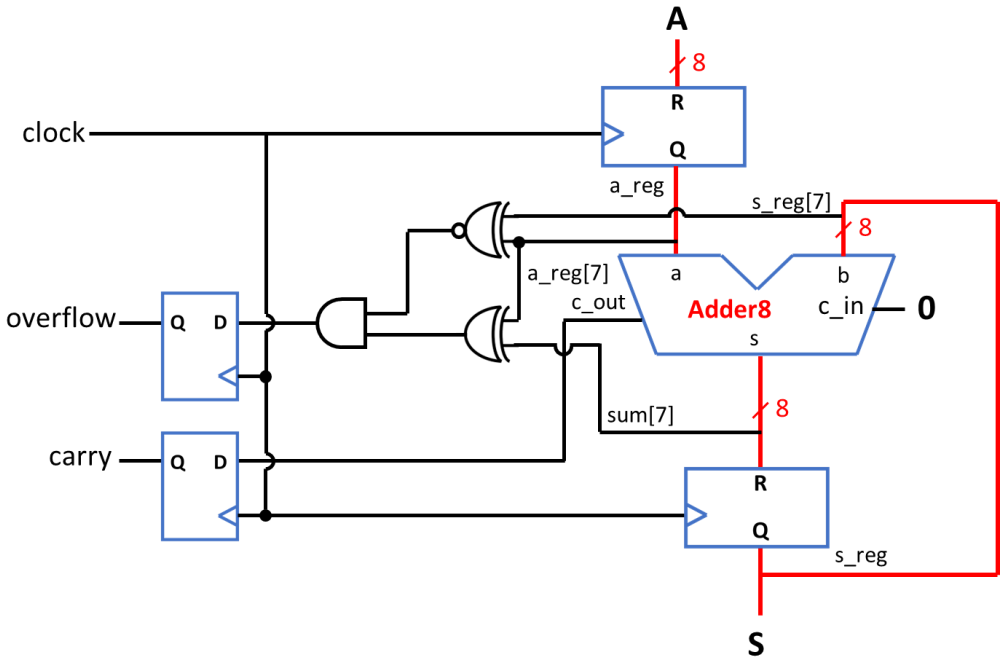
1.1. Thiết kế mạch logic:



Hình 1: Khối Full Adder



Hình 2: Bộ adder8 (cộng/trừ 2 số 8 bit)



Hình 3: Khối top-level bài Thí nghiệm 1

1.2. Hoạt động của mạch

Với các sơ đồ khối của mạch logic được mô tả ở phần 1, hoạt động của mạch được mô tả thông qua các khối chính như sau:

1.2.1. Khối thanh ghi (Register blocks):

- ***a_reg:*** Một thanh ghi 8-bit để lưu trữ giá trị đầu vào *i_A* tại mỗi sườn lên của xung clock.
- ***s_reg:*** Thanh ghi 8-bit chính của bộ tích lũy, dùng để lưu trữ kết quả của phép cộng (sum) từ chu kỳ trước. Giá trị của s_reg chính là đầu ra o_S của mạch.
- Cả hai thanh ghi đều được reset về 0 khi tín hiệu *i_reset* ở mức thấp.

1.2.2. Khối cộng 8-bit (8-bit adder):

Mạch sử dụng một bộ cộng 8-bit được định nghĩa trong module adder8. Module này được xây dựng từ 8 module full_adder (bộ cộng toàn phần 1-bit) kết nối nối tiếp với nhau.

Bộ cộng nhận hai đầu vào là a_reg (giá trị A hiện tại) và s_reg (tổng đã tích lũy) để tính ra tổng mới sum và bit nhớ cuối cùng c_out. Trong khối Adder8 có ngõ vào c_in để chọn chế độ (cộng/trừ) nhưng trong Thí nghiệm này chỉ thực hiện cộng nên c_in luôn được gán cho mức 0

1.2.3. Khối logic (Logic circuit):

Tính toán cờ nhớ (Carry Flag): Cờ nhớ carry_flag được gán trực tiếp bằng bit nhớ ra c_out của bộ cộng 8-bit. Nó sẽ lên mức 1 nếu phép cộng số không dấu 8-bit cho ra kết quả lớn hơn 255.

Tính toán cờ tràn (Overflow Flag): Cờ tràn overflow_flag được tính toán cho phép cộng số bù hai. Biểu thức logic assign overflow_flag = (~(a_reg[7]^s_reg[7])) & (a_reg[7]^sum[7]); thực hiện việc kiểm tra: cờ tràn sẽ lên mức 1 nếu hai số hạng có cùng dấu (bit MSB giống nhau) nhưng kết quả tổng lại có dấu khác.

1.2.4. Khối đầu ra (Output blocks):

Các tín hiệu cờ carry_flag và overflow_flag được đưa qua các flip-flop để đồng bộ với xung clock trước khi xuất ra ngoài thành o_carry và o_overflow.

Đầu ra o_S được gán bằng giá trị của thanh ghi tích lũy s_reg.

1.3. Code RTL (SystemVerilog)

Module full_adder:

```
module full_adder (
    input  logic a,
    input  logic b,
    input  logic c_in,
    output logic s,
    output logic c_out
);
    assign s     = a ^ b ^ c_in;
    assign c_out = (a & b) | (a & c_in) | (b & c_in);

endmodule
```

Module adder8:

```
module adder8(
    input  logic [7:0] a,
    input  logic [7:0] b,
    input  logic       c_in,
    output logic [7:0] s,
    output logic       c_out
);

   logic [8:0] carry_tmp;
   logic [7:0] temp;

   // Selection ADD or SUB
```

```
      assign temp = c_in ? ~b : b;

   // carry_tmp in
      assign carry_tmp[0] = c_in;

   //--------8 full-adder-----------//
      full_adder
fa0  (.a(a[0]),  .b(temp[0]),  .c_in(carry_tmp[0]),  .s(s[0]),  .c_out(carry_t
mp[1]));
      full_adder
fa1  (.a(a[1]),  .b(temp[1]),  .c_in(carry_tmp[1]),  .s(s[1]),  .c_out(carry_t
mp[2]));
      full_adder
fa2  (.a(a[2]),  .b(temp[2]),  .c_in(carry_tmp[2]),  .s(s[2]),  .c_out(carry_t
mp[3]));
      full_adder
fa3  (.a(a[3]),  .b(temp[3]),  .c_in(carry_tmp[3]),  .s(s[3]),  .c_out(carry_t
mp[4]));
      full_adder
fa4  (.a(a[4]),  .b(temp[4]),  .c_in(carry_tmp[4]),  .s(s[4]),  .c_out(carry_t
mp[5]));
      full_adder
fa5  (.a(a[5]),  .b(temp[5]),  .c_in(carry_tmp[5]),  .s(s[5]),  .c_out(carry_t
mp[6]));
      full_adder
fa6  (.a(a[6]),  .b(temp[6]),  .c_in(carry_tmp[6]),  .s(s[6]),  .c_out(carry_t
mp[7]));
      full_adder
fa7  (.a(a[7]),  .b(temp[7]),  .c_in(carry_tmp[7]),  .s(s[7]),  .c_out(carry_t
mp[8]));

      // carry_tmp out
      assign c_out = carry_tmp[8];

endmodule
```

Module exp1:

```
module exp1(
    input  logic [7:0] i_A,
    input  logic        i_clk,
    input  logic         i_reset,
    output logic [7:0] o_S,
    output logic         o_overflow,
    output logic         o_carry
);

    //================================
```

```
 //  Internal Signals
 //================================

 logic [7:0] a_reg;
 logic [7:0] s_reg;
logic [7:0] sum;
 logic       overflow_flag;
 logic       carry_flag;

 //================================
 //  Rister blocks
 //================================

always_ff @(posedge i_clk or negedge i_reset) begin
     if (!i_reset) begin
         a_reg <= 8'b0;
         s_reg <= 8'b0;
     end else begin
         a_reg <= i_A;
         s_reg <= sum;
     end
 end

 //================================
 //  8 bit adder
 //================================
 logic c_out;

 adder8 u_adder8 (
     .a(a_reg),
     .b(s_reg),
     .c_in(1'b0),
     .s(sum),
     .c_out(c_out)
 );

 //================================
 //  Logic circuit
 //================================

 assign carry_flag   = c_out;
 assign overflow_flag = (~(a_reg[7]^s_reg[7]))&(a_reg[7]^sum[7]);

 //================================
 //  Output blocks
 //================================
```

```
        always_ff @(posedge i_clk or negedge i_reset) begin
            if (!i_reset) begin
                o_carry <= 1'b0;
                o_overflow <= 1'b0;
            end else begin
                o_carry    <= carry_flag;
                o_overflow <= overflow_flag;
            end
        end

        assign o_S = s_reg;

endmodule
```

1.4. Kiểm định thiết kế:

1.4.1. Các giả định:

Trường hợp 1: Tích lũy thông thường (C=0, V=0):

- Thiết lập: Đầu vào A được gán giá trị 20 và mạch chạy trong 5 chu kỳ clock.
- Hoạt động: Mạch thực hiện phép cộng S = S + 20 liên tiếp.
- Dạng sóng dự kiến:

  Tín hiệu S sẽ tăng dần: 0 → 20 → 40 → 60 → 80 → 100.
- Các cờ o_carry và o_overflow luôn ở mức 0 vì phép cộng không gây tràn số bù hai và cũng không vượt quá giới hạn của số không dấu 8-bit.

Trường hợp 2: Kiểm tra cờ tràn (C=0, V=1):

- Thiết lập: Sau khi reset, đầu vào A được gán giá trị 80.
- Hoạt động: Ở chu kỳ clock đầu tiên, S = 0 + 80 = 80. Ở chu kỳ tiếp theo, mạch tính S = 80 + 80 = 160. Trong hệ bù hai 8-bit, 80 là số dương, nhưng 160 (nhị phân 10100000) lại biểu diễn cho số -96.
- Dạng sóng dự kiến:

  Tín hiệu S thay đổi từ 80 thành 160.

  Cờ o_overflow sẽ chuyển lên mức 1 ngay sau phép cộng 80+80 vì hai số dương cộng lại cho ra kết quả có dấu âm.

  Cờ o_carry vẫn ở mức 0.

Trường hợp 3: Kiểm tra cờ nhớ (C=1, V=0)

– Thiết lập: A được gán 180 để S nhận giá trị 180. Sau đó, A được gán 80.

– Hoạt động: Mạch tính S = 180 + 80 = 260. Kết quả này lớn hơn 255, giá trị lớn nhất của số không dấu 8-bit.

– Dạng sóng dự kiến:

Tín hiệu S sẽ hiển thị 4 (là 260 mod 256).

Cờ o_carry sẽ chuyển lên mức 1 để báo hiệu phép cộng đã vượt ngưỡng 255.

Cờ o_overflow vẫn ở mức 0 vì phép cộng được thực hiện giữa một số âm (180 tương đương -76 bù hai) và một số dương (80).

Trường hợp 4: Kiểm tra cả cờ nhớ và cờ tràn (C=1, V=1)

– Thiết lập: A được gán giá trị -100.

– Hoạt động: Ở chu kỳ đầu, S = 0 + (-100) = -100. Ở chu kỳ sau, mạch tính S = -100 + (-100) = -200. Giá trị -200 nằm ngoài khoảng [-128, 127] của số bù hai 8-bit.

– Dạng sóng dự kiến:

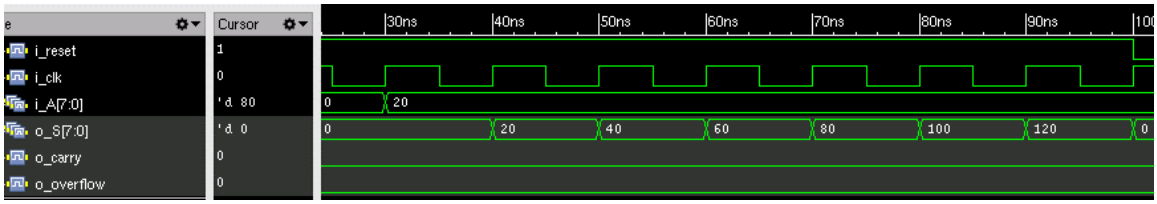
Tín hiệu S sẽ hiển thị giá trị 56 (giá trị wrap-around của -200).

Cờ o_overflow lên mức 1 vì hai số âm cộng lại cho ra kết quả dương.

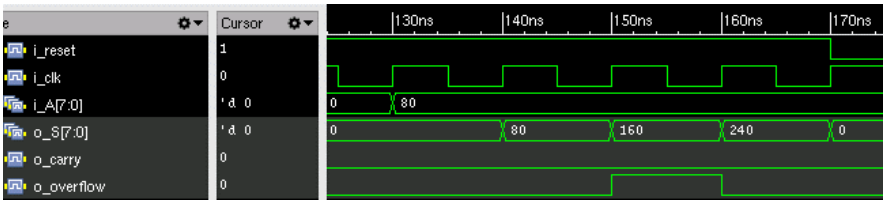
Cờ o_carry cũng lên mức 1 (bit nhớ ra từ MSB của phép cộng nhị phân).

1.4.2. Kết quả kiểm định:

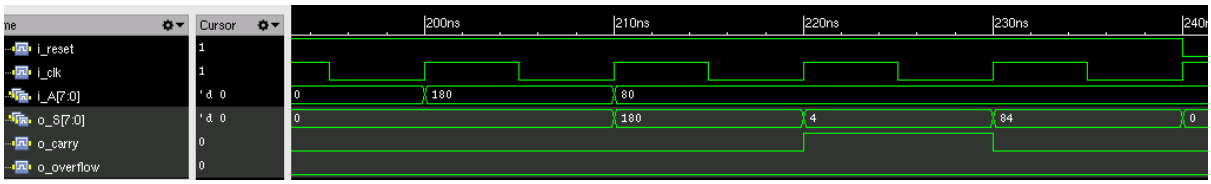
a) Dạng sóng (Các kết quả được hiển thị là số không dấu)



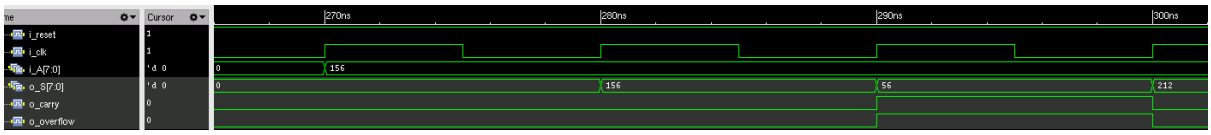
Trường hợp 1: Tích lũy thông thường (C=0, V=0)



Trường hợp 2: Kiểm tra cờ tràn (C=0, V=1)



Trường hợp 3: Kiểm tra cờ nhớ (C=1, V=0)



Trường hợp 4: Kiểm tra cả cờ nhớ và cờ tràn (C=1, V=1)

b) Transcript:

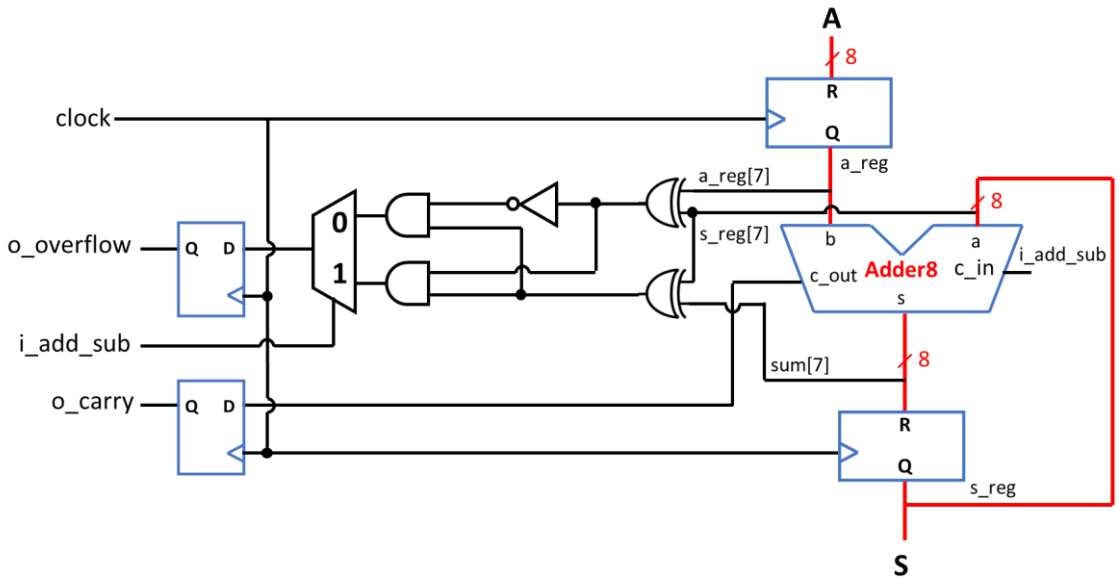
```
=================================================================
                              Starting Accumulator Test
=================================================================
[90 ns] Test Case: Add 20 for 5 clock cycle -> Normally accumulation: C = V = 0
S (Signed) =  100
S (Unsigned) = 100
[PASS] -> Sum is correct
[PASS] Carry flag is correct.
[PASS] Overflow flag is correct.
Result: C = 0 | V = 0
=================================================================
[160 ns] Test Case: Sum expected: 80 + 80 = 160
The result is out of range of 2's complement numbers -> V = 1
New S (Signed) =  -96
S (Unsigned) = 160
[PASS] -> Sum is correct
[PASS] Carry flag is correct.
[PASS] Overflow flag is correct.
Result: C = 0 | V = 1
=================================================================
[230 ns] Test Case: Sum expected: 180 (-76) + 80 = 260 (4)
The result is beyond the range of an unsigned 8-bit number -> C = 1
New S (Unsigned) =   4
S (Signed) =    4
[PASS] -> Sum is correct
[PASS] Carry flag is correct.
[PASS] Overflow flag is correct.
Result: C = 1 | V = 0
=================================================================
[300 ns] Test Case: Sum expected: 156 (-100) + 156 (-100) = 312 (-200)
The result is both out of range of 2's complement numbers and beyond the range of an unsigned 8-bit number
New S (Signed) =   56
New S (Unsigned) =  56
[PASS] -> Sum is correct
[PASS] Carry flag is correct.
[PASS] Overflow flag is correct.
Result: C = 1 | V = 1
=================================================================
                              Test Finished
=================================================================
```

THÍ NGHIỆM 2:

Objective: Know how to program a system to add or subtract the value of an input A to itself repeatedly.

2.1. Thiết kế mạch logic:

Các khối Full-Adder và Adder8 được tái sử dụng từ Thí nghiệm 1.



Hình 4: Khối top-level bài Thí nghiệm 2

2.2. Hoạt động của mạch

Mạch được phát triển từ bộ tích lũy ở Thí nghiệm 1 với những thay đổi để hỗ trợ chức năng trừ:

2.2.1. Tín hiệu điều khiển i_add_sub:

Một đầu vào mới i_add_sub được thêm vào để chọn chế độ hoạt động.

Khi i_add_sub = 0, mạch hoạt động như một bộ tích lũy cộng thông thường

Khi i_add_sub = 1, mạch thực hiện phép trừ (S = S - A).

2.2.2. Cải tiến Khối cộng 8-bit:

Phép trừ được hiện thực bằng phương pháp bù hai: S - A = S + (~A) + 1.

Tín hiệu i_add_sub được nối trực tiếp vào chân c_in của module adder8.

Bên trong module adder8, khi c_in (tức i_add_sub) bằng 1, toán hạng b (tức a_reg) sẽ bị đảo bit (~b) và bit c_in cũng đóng vai trò là +1 trong công thức bù hai. Khi c_in bằng 0, b được giữ nguyên và mạch thực hiện phép cộng bình thường.

2.2.3. Cải tiến Khối logic:

Cờ nhớ/mượn (Carry/Borrow Flag): Khi trừ (i_add_sub = 1), cờ c_out từ bộ cộng có ý nghĩa là cờ không mượn (not-borrow). Nó sẽ bằng 1 nếu S ≥ A (phép trừ không cần mượn) và bằng 0 nếu S < A (phép trừ cần mượn).

Cờ tràn (Overflow Flag): Biểu thức logic cho cờ tràn được mở rộng để xử lý cả hai trường hợp: *Khi cộng (i_add_sub = 0):* Logic giữ nguyên như Thí nghiệm 1; Khi trừ (i_add_sub = 1): Logic là (a_reg[7]^s_reg[7]) & (s_reg[7]^sum[7]). Cờ tràn sẽ lên 1 nếu phép trừ được thực hiện giữa hai số có dấu khác nhau và kết quả lại có dấu khác với số bị trừ (s_reg).

2.3. Code RTL (SystemVerilog)

Các module *adder8, full_adder* đã được liệt kê ở Thí nghiệm 1.

Module exp2:

```
module exp2(
    input  logic [7:0] i_A,
    input  logic        i_clk,
    input  logic         i_reset,
    input  logic         i_add_sub,
    output logic [7:0] o_S,
    output logic         o_overflow,
    output logic         o_carry
);

    //================================
    //  Internal Signals
    //================================

    logic [7:0] a_reg;
    logic [7:0] s_reg;
   logic [7:0] sum;
    logic       overflow_flag;
    logic       carry_flag;

    //================================
    //  Rister blocks
    //================================
```

```
    always_ff @(posedge i_clk or negedge i_reset) begin
        if (!i_reset) begin
            a_reg <= 8'b0;
            s_reg <= 8'b0;
        end else begin
            a_reg <= i_A;
            s_reg <= sum;
        end
    end

    //================================
    //  8 bit adder
    //================================
    logic c_out;

    adder8 u_adder8 (
        .a(s_reg),
        .b(a_reg),
        .c_in(i_add_sub),
        .s(sum),
        .c_out(c_out)
    );

    //================================
    //  Logic circuit
    //================================

    assign carry_flag   = c_out;
    assign overflow_flag = i_add_sub?
(a_reg[7]^s_reg[7])&(s_reg[7]^sum[7]):(~(a_reg[7]^s_reg[7]))&(a_reg[7]^sum[7])
;

    //================================
    //  Output blocks
    //================================

        always_ff @(posedge i_clk or negedge i_reset) begin
            if (!i_reset) begin
                o_carry <= 1'b0;
                o_overflow <= 1'b0;
            end else begin
                o_carry    <= carry_flag;
                o_overflow <= overflow_flag;
            end
        end

        assign o_S = s_reg;
```

```
endmodule
```

2.4. Kiểm định thiết kế:

2.4.1. Các giả định:

Trường hợp 0: Cộng tích lũy thông thường (C=0, V=0)

Trường hợp này đã được phân tích trong bài Thí nghiệm 1, vì vậy nhóm chỉ thử nghiệm lại việc cộng tích lũy cơ bản, và sẽ phân tích sâu vào các trường hợp trừ tích lũy

Trường hợp 1: Trừ thông thường (Không mượn, C=1, V=0)

- Thiết lập: S được nạp giá trị 100. Sau đó, A được gán 30 và add_sub được bật lên 1. Mạch chạy trong 3 chu kỳ.
- Hoạt động: Mạch thực hiện S = S - 30 liên tiếp.
- Dạng sóng dự kiến:

  S sẽ giảm dần: 100 → 70 → 40 → 10.

  Vì S luôn lớn hơn A, o_carry (cờ không mượn) sẽ luôn ở mức 1.

  o_overflow ở mức 0.

Trường hợp 2: Trừ có mượn (C=0, V=0)

- Thiết lập: Tiếp nối trường hợp trên, S đang bằng 10 và A vẫn là 30.
- Hoạt động: Mạch thực hiện phép tính S = 10 - 30 = -20.
- Dạng sóng dự kiến:

  S chuyển từ 10 thành -20 (nhị phân 11101100).

  Vì 10 < 30, phép trừ số không dấu cần mượn, do đó cờ o_carry sẽ xuống mức 0.

  o_overflow vẫn ở mức 0 vì không có tràn trong phép toán bù hai này.

Trường hợp 3: Tràn có mượn khi trừ (C=0, V=1)

- Thiết lập: S được nạp 100. Sau đó, A được gán -50 và add_sub bật 1.
- Hoạt động: Mạch tính S = 100 - (-50) = 150. Kết quả 150 nằm ngoài khoảng [-128, 127] của số bù hai 8-bit.
- Dạng sóng dự kiến:

  S sẽ hiển thị 150 (tương đương -106 trong hệ bù hai).

o_overflow sẽ lên mức 1 vì lấy số dương (100) trừ đi số âm (-50) đã gây ra kết quả vượt quá giới hạn dương.

o_carry sẽ ở mức 0.

Trường hợp 4: Tràn không mượn khi trừ (C=1, V=1)

– Thiết lập: S được nạp -100. Sau đó, A được gán 50 và add_sub bật 1.

– Hoạt động: Mạch tính S = -100 - 50 = -150. Kết quả nằm ngoài khoảng [-128, 127].

– Dạng sóng dự kiến:

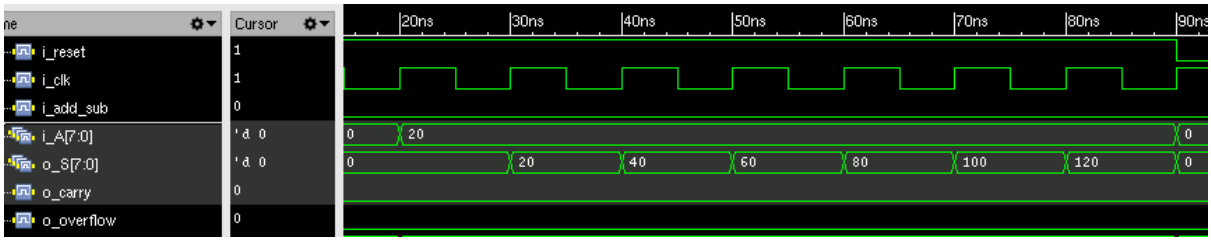
S sẽ hiển thị 106 (giá trị wrap-around của -150).

o_overflow sẽ lên mức 1 vì lấy số âm (-100) trừ đi số dương (50) đã gây ra kết quả vượt quá giới hạn âm.

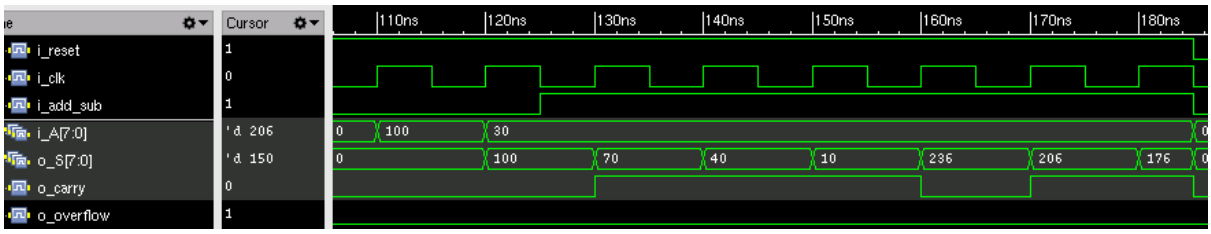
o_carry sẽ ở mức 1 (bit cờ không mượn).

2.4.2. Kết quả kiểm định:

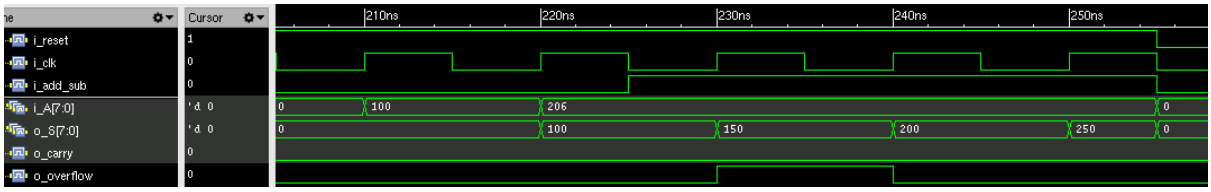
a) Dạng sóng (Các kết quả được hiển thị là số không dấu)



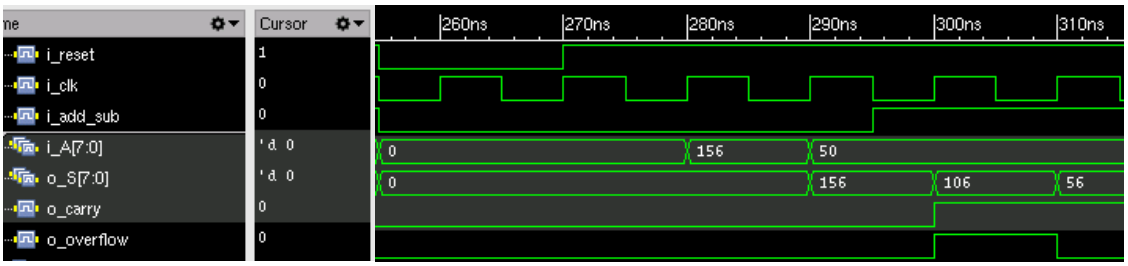
Trường hợp 0: Cộng tích lũy thông thường (C=0, V=0)



Trường hợp 1 và 2: Trừ thông thường (Không mượn, C=1, V=0) và có mượn (C=0, V=0)



Trường hợp 3: Tràn có mượn khi trừ (C=0, V=1)



Trường hợp 4: Tràn không mượn khi trừ (C=1, V=1)

b) Transcript:

```
=================================================================
         Starting Add Accumulator Test
=================================================================
[71 ns] Test Case: Add 20 for 5 clock cycles -> Normally accumulation: C = V = 0
S (Signed) =  100
S (Unsigned) = 100
[PASS] -> Sum is correct
[PASS] -> Carry flag is correct.
[PASS] -> Overflow flag is correct.
Result: C = 0 | V = 0
```

```
=====================================================================
         Starting Sub Accumulator Test
=====================================================================
[156 ns] Test Case: Normal sub, no borrow: S = 100 - 30 for 3 cycles
Subtraction is normal and does not require borrowing -> C = 1
S (Unsigned) =  10
S (Signed) =   10
[PASS] -> Sum is correct
[PASS] -> Carry flag is correct.
[PASS] -> Overflow flag is correct.
Result: C = 1 | V = 0
=====================================================================
[166 ns] Test Case: S is less than A, borrow in unsigned: C = 0
S < A (borrow)
S (Signed) =  -20
New S (Unsigned) = 236
[PASS] -> Sum is correct
[PASS] -> Carry flag is correct.
[PASS] -> Overflow flag is correct.
Result: C = 0 | V = 0
=====================================================================
[236 ns] Test Case: Result expected: 100 - (-50) = 150
The result is out of range of 2's complement numbers and S < A (borrow)
New S (Signed) = -106
New S (Unsigned) = 150
[PASS] -> Sum is correct
[PASS] -> Carry flag is correct.
[PASS] -> Overflow flag is correct.
Result: C = 0 | V = 1
=====================================================================
[306 ns] Test Case: Result expected: -100 - 50 = -150
The result exceeds the range of 2's complement numbers and S doesn't borrow
New S (Signed) =  106
S (Unsigned) = 106
[PASS] -> Sum is correct
[PASS] -> Carry flag is correct.
[PASS] -> Overflow flag is correct.
Result: C = 1 | V = 1
=====================================================================
                Test Finished
=====================================================================
```

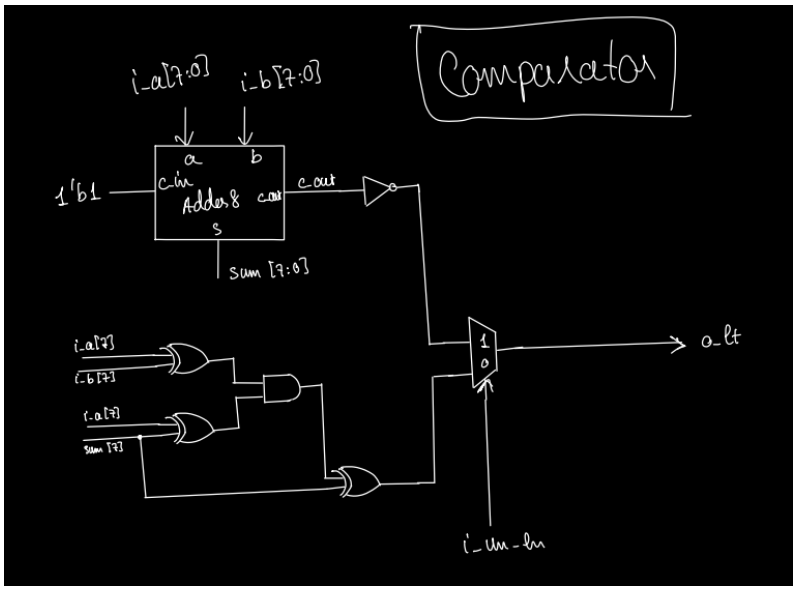
THÍ NGHIỆM 3:

Objective: Know how to design a circuit that finds the larger and smaller numbers between two 8-bit numbers.

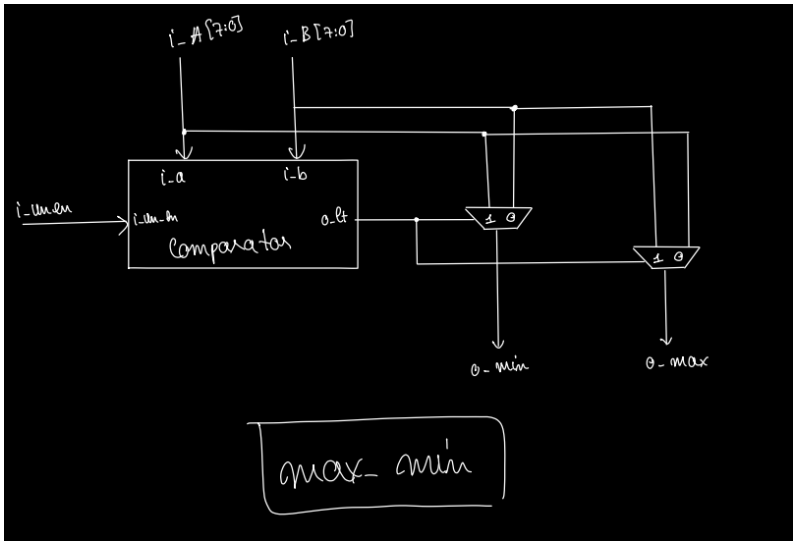
Description: The circuit in Figure 3 is an example of a design that determines the larger and smaller values between two 8-bit numbers. The circuit is built using one comparator and two multiplexers. The comparator provides the select signal for the multiplexers' outputs.

3.1. Thiết kế mạch logic:

Các khối Full-Adder và Adder8 được tái sử dụng từ Thí nghiệm 1.



Hình 4: Bên trong khối Comparator (Bộ so sánh 2 số 8 bit)



Hình 5: Khối max_min – top-level của bài Thí nghiệm 3

3.2. Hoạt động của mạch

Cách so sánh bên trong comparator (dựa trên phép trừ hai bù 2): Comparator không dùng toán tử so sánh mà thực hiện phép trừ: Tính S = A − B = A + (~B) + 1 bằng chuỗi full-adder 8 bit (ripple-carry). Lấy c_out là carry ra cuối cùng của phép cộng và S[7] là bit dấu của kết quả.

Với không dấu (unsigned): "A < B" tương đương có mượn trong phép trừ. Vì mạch cộng bù-2 dùng carry-out thay cho borrow, ta có AltB_unsigned = ~c_out.

Với có dấu (two's complement): kiểm tra tràn dấu

ov = (A[7] ⊕ B[7]) · (A[7] ⊕ S[7]).

Khi đó quan hệ "A < B (signed)" được quyết định bởi AltB_signed = S[7] ⊕ ov.

Tín hiệu điều khiển sign (i_un_en) chọn chế độ so sánh: AltB = chọn(AltB_unsigned, AltB_signed) theo chế độ. Tạo Min và Max bằng hai MUX 2:1. Tín hiệu AltB đi ra comparator được đưa đến chân sel của hai MUX 2:1: MUX Min nhận ngõ vào A và B. Nếu AltB = 1 (A < B) thì Min = A; ngược lại Min = B. MUX Max nhận ngõ vào B và A. Nếu AltB = 1 (A < B) thì Max = B; ngược lại Max = A.

3.3. Code RTL (SystemVerilog)

Các module *adder8, full_adder* đã được liệt kê ở Thí nghiệm 1.

Module comparator:

```
module comparator(
    input  logic [7:0] i_a,
    input  logic [7:0] i_b,
    input  logic         i_un_en,   //1: Unsigned , 0: Signed
    output logic         o_lt
);
    // --- Internal Signals ---
     logic [7:0] sum;
     logic       c_out;
     logic       a_sign, b_sign, sum_sign;
     logic       overflow;
     logic       unsigned_less, signed_less;

    // --- Sign bits ---
    assign a_sign = i_a[7];
    assign b_sign = i_b[7];

    // --- Subtraction using the 8-bit adder ---
```

```
    adder8 u_subtractor (
    .a     (i_a),
    .b     (i_b),
    .c_in  (1'b1),
    .s     (sum),
    .c_out (c_out)
    );

    assign sum_sign = sum[7];

     // --- Less Than Check ---

    // 1. Unsigned comparison (i_un_en = 1)
    assign unsigned_less = ~c_out;

     // 2. Signed comparison (i_un_en = 0)
    assign overflow    = (a_sign ^ b_sign) & (a_sign ^ sum_sign);
    assign signed_less = sum_sign ^ overflow;

     // 3. Final Selection
    assign o_lt = i_un_en ? unsigned_less : signed_less;

endmodule
```

Module exp3:

```
module exp3(
    input  logic [7:0] i_A,
    input  logic [7:0] i_B,
    input  logic         i_un_en,
    output logic [7:0] o_min,
    output logic [7:0] o_max
);

    // --- Internal Signals --- //
    logic   AltB;

    comparator u_comp(
        .i_a(i_A),
        .i_b(i_B),
        .i_un_en(i_un_en),
        .o_lt(AltB)
    );

    // --- Output --- //
    assign o_min = AltB ? i_A : i_B;
    assign o_max = AltB ? i_B : i_A;
endmodule
```

3.4. Kiểm định thiết kế:

3.4.1. Các giả định:

Hệ thống có khả năng xử lý cả số không dấu (unsigned) và số có dấu (signed) thông qua tín hiệu điều khiển i_un_en. Các trường hợp kiểm thử được chia thành hai nhóm chính.

So sánh không dấu (Unsigned Comparison, i_un_en = 1)

Trường hợp 1a (A < B): Đầu vào i_A = 10 và i_B = 20. Mạch hoạt động đúng khi xác định o_min = 10 và o_max = 20.

Trường hợp 1b (A > B): Đầu vào i_A = 30 và i_B = 15. Kết quả đầu ra chính xác là o_min = 15 và o_max = 30.

Trường hợp 1c (A = B): Khi hai đầu vào bằng nhau i_A = 50, i_B = 50 , mạch đưa ra kết quả o_min = 50 và o_max = 50.

Trường hợp 1d (Cận biên): Kiểm tra với giá trị lớn nhất và nhỏ nhất của số không dấu 8-bit, i_A = 255 (8'hFF) và i_B = 0 (8'h00). Mạch so sánh và cho ra o_min = 0 và o_max = 255.

So sánh có dấu (Signed Comparison, i_un_en = 0)

Trường hợp 2b, 2c (Số dương và số âm): Hệ thống được đưa vào một số âm và một số dương. Cụ thể, khi i_A = -5 và i_B = 5, mạch xác định o_min = -5, o_max = 5. Ngược lại, khi i_A = 5 và i_B = -5, kết quả là o_min = -5, o_max = 5.

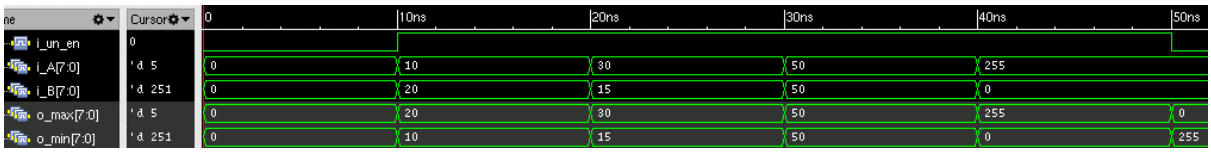
Trường hợp 2d, 2e (Hai số âm): Hệ thống so sánh hai số âm với nhau, ví dụ i_A = -10 và i_B = -5. Kết quả đầu ra là o_min = -10 và o_max = -5.

Trường hợp 2h (Cận biên): Đây là trường hợp kiểm tra quan trọng nhất của so sánh có dấu, so sánh số dương lớn nhất (127, 7FH) với số âm nhỏ nhất (-128, 80H). Kết quả là o_min = -128 và o_max = 127.

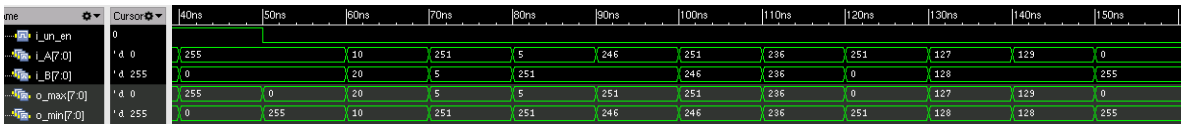
Trường hợp 2j (So sánh -1 với 0): Hệ thống so sánh 0 với -1 (FFH). Kết quả o_min = -1 và o_max = 0.

3.4.2. Kết quả kiểm định:

a) *Dạng sóng (Các kết quả được hiển thị là số không dấu):*



Trường hợp 1: So sánh 2 số không dấu (i_un_en = 1)



Trường hợp 2: So sánh 2 số có dấu (i_un_en = 0)

b) Transcript:

```
=================================================================
        Starting Comparison Test
=================================================================
        1/ Starting Unsigned Comparison Test
=================================================================
[20 ns] Test 1a: A < B, A =  10, B =  20
[PASS] -> Min =  10, Max =  20
=================================================================
[30 ns] Test 1b: A > B, A =  30, B =  15
[PASS] -> Min =  15, Max =  30
=================================================================
[40 ns] Test 1c: A = B, A =  50, B =  50
[PASS] -> Min =  50, Max =  50
=================================================================
[50 ns] Test 1d: Edge case (255 > 0), A = 255, B =   0
[PASS] -> Min =   0, Max = 255
=================================================================
```

```
         2/ Starting Signed Comparison Test
=====================================================================
[70 ns] Test: Test 2a: A < B (pos, pos), A =   10, B =   20
[PASS] -> Min =   10, Max =   20
=====================================================================
[80 ns] Test: Test 2b: A < B (neg, pos), A =   -5, B =    5
[PASS] -> Min =   -5, Max =    5
=====================================================================
[90 ns] Test: Test 2c: A > B (pos, neg), A =    5, B =   -5
[PASS] -> Min =   -5, Max =    5
=====================================================================
[100 ns] Test: Test 2d: A < B (neg, neg), A =  -10, B =   -5
[PASS] -> Min =  -10, Max =   -5
=====================================================================
[110 ns] Test: Test 2e: A > B (neg, neg), A =   -5, B =  -10
[PASS] -> Min =  -10, Max =   -5
=====================================================================
[120 ns] Test: Test 2f: A == B (neg, neg), A =  -20, B =  -20
[PASS] -> Min =  -20, Max =  -20
=====================================================================
[130 ns] Test: Test 2g: Compare with 0, A =   -5, B =    0
[PASS] -> Min =   -5, Max =    0
=====================================================================
[140 ns] Test: Test 2h: Edge case 1, A =  127, B = -128
[PASS] -> Min = -128, Max =  127
=====================================================================
[150 ns] Test: Test 2i: Compare nearly negs, A = -127, B = -128
[PASS] -> Min = -128, Max = -127
=====================================================================
[160 ns] Test: Test 2j: Compare 0 with -1, A =    0, B =   -1
[PASS] -> Min =   -1, Max =    0
=====================================================================
                Test Finished
=====================================================================
```

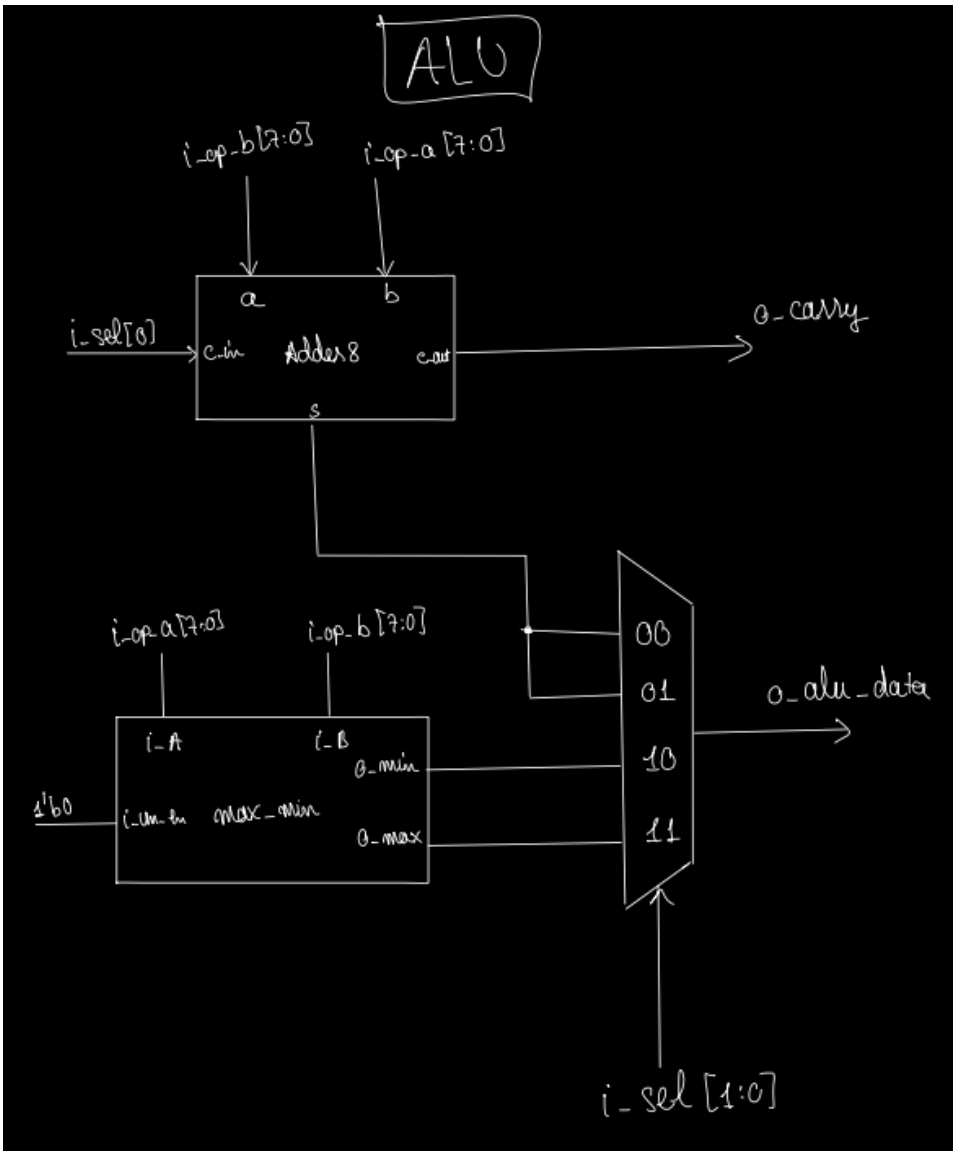
THÍ NGHIỆM 4:

Objective: Know how to program ALU circuit with registered inputs and outputs.

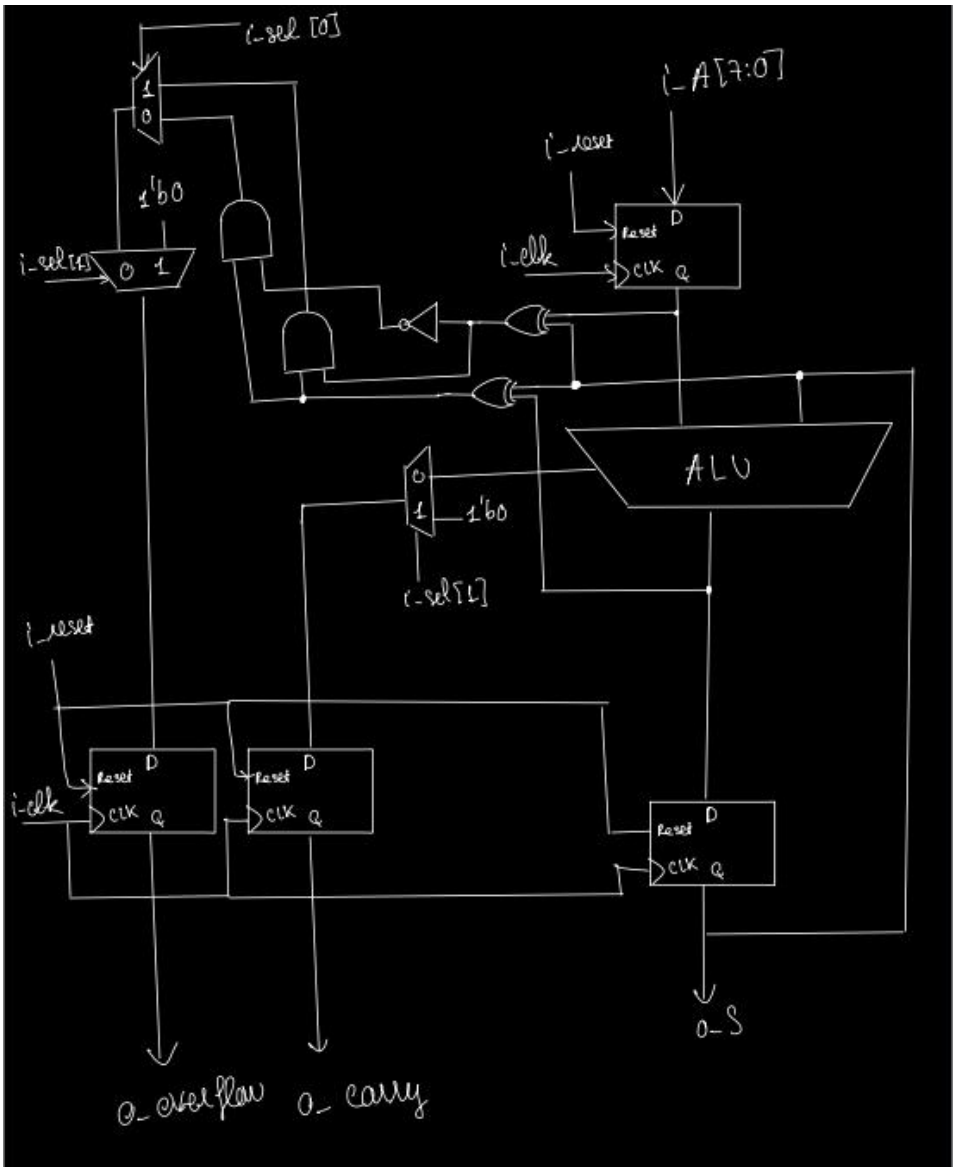
Desription: The circuit in Figure 4 shows an ALU circuit with registered inputs and outputs. The ALU can implement add, subtract, find maximum and minimum depending on *Sel* input which is shown in Table 1. Write System Verilog code to implement this system.

4.1. Thiết kế mạch logic:

Các khối Full-Adder, Adder8, Comparator và Max_min được tái sử dụng từ Thí nghiệm 1 và 3



Hình 6: Bên trong khối ALU



Hình 7: Module top-level

4.2. Hoạt động của mạch

Mạch sẽ thực hiện các phép toán theo tín hiệu Sel ngõ vào như sau:

Sel Input	ALU operator
00	Add
01	Subtract
10	Maximum
11	Minimum

Table 1: Operation of ALU circuit.

Tại mỗi chu kì, ALU thực hiện các phép ADD, SUB, MAX, MIN giữa 2 giá trị A (input) và S (kết quả phép toán ở chu kì trước), sau đó báo các cờ overflow và carry

(overflow và carry luôn bằng 0 với MAX, MIN). Lưu ý: Phép so sánh mặc định so sánh giữa 2 số có dấu.

4.3. Code RTL (SystemVerilog)

Các module *adder8, full_adder, comparator* đã được liệt kê ở Thí nghiệm 1 và 3.

Module max_min:

```
module max_min(
    input  logic [7:0] i_A,
    input  logic [7:0] i_B,
    input  logic         i_un_en,
    output logic [7:0] o_min,
    output logic [7:0] o_max
);

    // --- Internal Signals --- //
    logic   AltB;

    comparator u_comp(
        .i_a(i_A),
        .i_b(i_B),
        .i_un_en(i_un_en),
        .o_lt(AltB)
    );

    // --- Output --- //
    assign o_min = AltB ? i_A : i_B;
    assign o_max = AltB ? i_B : i_A;
endmodule
```

Module alu:

```
module alu(
    input  logic [7:0] i_op_a,
    input  logic [7:0] i_op_b,
    input  logic [1:0] i_sel,
    output logic [7:0] o_alu_data,
    output logic         o_carry
);

    // ALU encoding
    localparam ADD = 2'b00;
    localparam SUB = 2'b01;
    localparam MAX = 2'b10;
    localparam MIN = 2'b11;
```

```
    //==================================================
    // 8-bit Adder/Subtractor
    //==================================================
    logic [7:0] add_sub_result;

     adder8 u_add_sub (
         .a     (i_op_b),
         .b     (i_op_a),
         .c_in  (i_sel[0]),  // 0: ADD, 1: SUB
         .c_out (o_carry),
         .s   (add_sub_result)
     );

     //==================================================
    // Determine MAX and MIN
    //==================================================
     logic [7:0] max_result;
     logic [7:0] min_result;

     max_min u_maxmin(
         .i_A(i_op_a),
         .i_B(i_op_b),
         .i_un_en(1'b0),
         .o_min(min_result),
         .o_max(max_result)
     );

     //==================================================
    // ALU Mux
    //==================================================
     always_comb begin
         unique case (i_sel)
                 ADD: o_alu_data = add_sub_result;
                 SUB: o_alu_data = add_sub_result;
                 MAX: o_alu_data = max_result;
                 MIN: o_alu_data = min_result;
                 default: o_alu_data = 8'b0;
         endcase
     end

endmodule
```

Module exp4:

```
module exp4(
    input  logic [7:0] i_A,
    input  logic        i_clk,
    input  logic         i_reset,
```

```
    input  logic [1:0]       i_sel,
    output logic [7:0] o_S,
    output logic         o_overflow,
    output logic         o_carry
);

   //================================
    //  Internal Signals
    //================================

    logic [7:0] a_reg;
    logic [7:0] s_reg;
   logic [7:0] alu_data;
    logic       overflow_flag;
    logic       carry_flag;

   //================================
    //  Rister blocks
    //================================

   always_ff @(posedge i_clk or negedge i_reset) begin
            if (!i_reset) begin
                a_reg <= 8'b0;
                s_reg <= 8'b0;
            end else begin
                a_reg <= i_A;
                s_reg <= alu_data;
            end
    end

   //================================
    //  ALU implement
    //================================
    logic   c_out;

    alu u_alu(
        .i_op_a(a_reg),
        .i_op_b(s_reg),
        .i_sel(i_sel),
        .o_alu_data(alu_data),
        .o_carry(c_out)
    );

    //================================
    //  Logic circuit
    //================================
```

```
    always_comb begin
            if (i_sel[1]) begin
                carry_flag    = 1'b0;
                overflow_flag = 1'b0;
            end else begin
                carry_flag    = c_out;
                overflow_flag = i_sel[0]?
(a_reg[7]^s_reg[7])&(s_reg[7]^alu_data[7]):(~(a_reg[7]^s_reg[7]))&(a_reg[7]^al
u_data[7]);
            end
    end

    //================================
    //  Output blocks
    //================================

    always_ff @(posedge i_clk or negedge i_reset) begin
            if (!i_reset) begin
                o_carry <= 1'b0;
                o_overflow <= 1'b0;
            end else begin
                o_carry    <= carry_flag;
                o_overflow <= overflow_flag;
            end
    end

    assign o_S = s_reg;
endmodule
```

4.4. Kiểm định thiết kế:

4.4.1. Các giả định:

Hệ thống kiểm định được xây dựng để xác minh tuần tự các chức năng của ALU, bao gồm cộng, trừ, tìm max và tìm min, thông qua việc thay đổi tín hiệu sel. Dạng sóng ghi lại một chuỗi các phép toán, trong đó kết quả S của phép toán trước thường được dùng làm một trong hai toán hạng cho phép toán kế tiếp.

Chức năng cộng (ADD, sel=00)

Kiểm tra cơ bản và tràn số có dấu: Chuỗi phép cộng bắt đầu từ 0 + 25 = 25, tiếp theo là 25 + 30 = 55. Sau đó, phép cộng 55 + 100 = 155 làm cho cờ tràn o_overflow bật lên 1 vì 155 vượt quá giới hạn 127 của số có dấu 8-bit.

Kiểm tra tràn và nhớ: Phép toán 155 + 150 = 305 làm cho cả hai cờ o_overflow và o_carry đều bật lên 1. Kết quả o_S là 49 (305 – 256), đúng như mong đợi. Các dạng sóng này xác nhận logic xử lý cờ của phép cộng hoạt động chính xác.

Chức năng trừ (SUB, sel=01)

Kiểm tra tràn số có dấu: Dạng sóng cho thấy phép trừ -81 - 50 cho kết quả -131. Vì -131 nhỏ hơn giới hạn -128, cờ o_overflow đã được bật lên 1 để báo hiệu lỗi tràn.

Kiểm tra tràn và cờ mượn (borrow): Phép trừ 105 - (-100) tương đương với 105 + 100 = 205. Kết quả này gây tràn số có dấu (V=1) và cũng cần mượn trong phép trừ số không dấu (C=0). Dạng sóng đã thể hiện đúng các giá trị cờ này.

Chức năng tìm Max/Min (sel=10/sel=11)

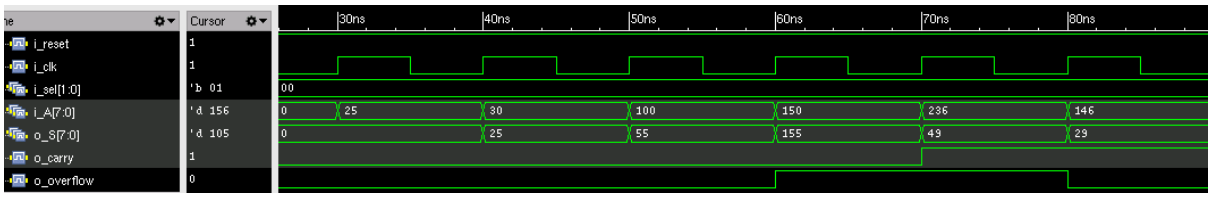
Kiểm tra MAX: Khi sel = 2'b10, mạch thực hiện tìm số lớn hơn. Ví dụ, khi so sánh A = 10 và S = -15, đầu ra o_S là 10. Khi hai toán hạng bằng nhau (-5 và -5), đầu ra vẫn là -5.

Kiểm tra MIN: Khi sel = 2'b11, mạch tìm số nhỏ hơn. Ví dụ, khi so sánh A = 20 và S = 10, đầu ra o_S là 10. Khi so sánh A = -5 và S = 50, đầu ra o_S là -5.

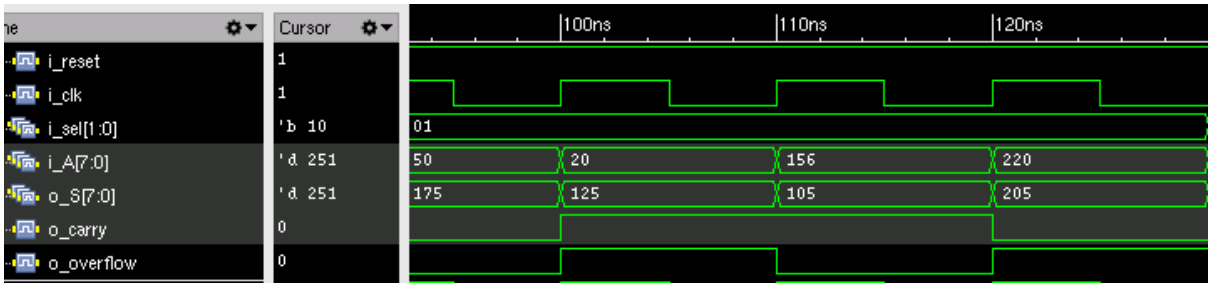
Trong tất cả các trường hợp kiểm tra MAX/MIN, các cờ o_carry và o_overflow luôn ở mức 0, vì các phép toán này không ảnh hưởng đến chúng.

4.4.2. Kết quả kiểm định:

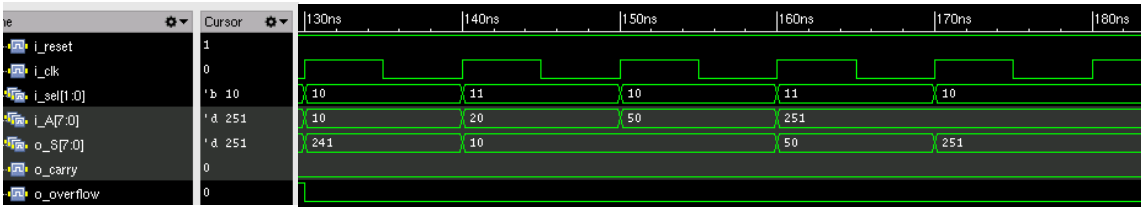
a) Dạng sóng (Các kết quả được hiển thị là số không dấu):



Chức năng cộng (ADD, sel=00)



Chức năng trừ (SUB, sel=01)



Chức năng tìm Max/Min (sel=10/sel=11)

b) Transcript:

```
=========================================================
======            STARTING ALU TB                ======
=========================================================
                      ADD TEST
=========================================================
[51 ns] TEST: ADD Pos+Pos
[Signed]   A =   30, S_in =   25
[Unsigned] A =  30 , S_in =  25
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   55
S (Unsigned) =  55
=========================================================
[61 ns] TEST: ADD Signed Overflow
[Signed]   A =  100, S_in =   55
[Unsigned] A = 100 , S_in =  55
[PASS] -> Flag result: C = 0 | V = 1
S (Signed) = -101
S (Unsigned) = 155
=========================================================
[71 ns] TEST: ADD Signed Overflow & Unsigned Carry
[Signed]   A = -106, S_in = -101
[Unsigned] A = 150 , S_in = 155
[PASS] -> Flag result: C = 1 | V = 1
S (Signed) =   49
S (Unsigned) =  49
=========================================================
[81 ns] TEST: ADD Carry Unsigned
[Signed]   A =  -20, S_in =   49
[Unsigned] A = 236 , S_in =  49
[PASS] -> Flag result: C = 1 | V = 0
S (Signed) =   29
S (Unsigned) =  29
=========================================================
[91 ns] TEST: ADD Basic
[Signed]   A = -110, S_in =   29
[Unsigned] A = 146 , S_in =  29
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =  -81
S (Unsigned) = 175
```

```
=========================================================
                    SUB TEST (S - A)
=========================================================
[101 ns] TEST: SUB Signed Overflow, no borrow
[Signed]   A =   50, S_in =  -81
[Unsigned] A =  50 , S_in = 175
[PASS] -> Flag result: C = 1 | V = 1
S (Signed) =  125
S (Unsigned) = 125
=========================================================
[111 ns] TEST: SUB Basic, no borrow
[Signed]   A =   20, S_in =  125
[Unsigned] A =  20 , S_in = 125
[PASS] -> Flag result: C = 1 | V = 0
S (Signed) =  105
S (Unsigned) = 105
=========================================================
[121 ns] TEST: SUB Pos-Neg Overflow, borrow
[Signed]   A = -100, S_in =  105
[Unsigned] A = 156 , S_in = 105
[PASS] -> Flag result: C = 0 | V = 1
S (Signed) =  -51
S (Unsigned) = 205
=========================================================
[131 ns] TEST: SUB Neg-Neg, borrow
[Signed]   A =  -36, S_in =  -51
[Unsigned] A = 220 , S_in = 205
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =  -15
S (Unsigned) = 241
```

```
=======================================================
                      MAX/MIN TEST
=======================================================
[141 ns] TEST: MAX (A > S)
[Signed]   A =   10, S_in =  -15
[Unsigned] A =  10 , S_in = 241
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   10
S (Unsigned) =  10
=======================================================
[151 ns] TEST: MIN (A > S)
[Signed]   A =   20, S_in =   10
[Unsigned] A =  20 , S_in =  10
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   10
S (Unsigned) =  10
=======================================================
[161 ns] TEST: MAX (A > S)
[Signed]   A =   50, S_in =   10
[Unsigned] A =  50 , S_in =  10
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   50
S (Unsigned) =  50
=======================================================
[171 ns] TEST: MIN (A < S)
[Signed]   A =   -5, S_in =   50
[Unsigned] A = 251 , S_in =  50
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   -5
S (Unsigned) = 251
=======================================================
[181 ns] TEST: MAX (A = S)
[Signed]   A =   -5, S_in =   -5
[Unsigned] A = 251 , S_in = 251
[PASS] -> Flag result: C = 0 | V = 0
S (Signed) =   -5
S (Unsigned) = 251
=======================================================
======            SIMULATION ENDED              ======
=======================================================
```